

Practical - 6

Aim : To implement the Chain Matrix Multiplication problem using Dynamic Programming and determine the minimum number of scalar multiplications required to multiply a sequence of matrices.

Algorithm :

Algorithm:

Input: Number of matrices n
Output: Array of dimensions $P[0 \dots n]$

1. Start
2. Read the numbers of matrices n
3. Read the dimensions array $P[0 \dots n]$
4. Initialize $C[i][i] = 0$ for all matrices
5. for $d = 1$ to $n - 1$:
 for $i = 1$ to $n - d$:
 Set $j = i + d$
 Set $min = \infty$
 for $k = i$ to $j - 1$:
 Calculate:
 $q = C[i][k] + C[k+1][j]$
 if $q < min$, update $min = q$
 Set $C[i][j] = min$
6. Print $C[1][n]$ as the minimum multiplication cost.
7. Stop

Program / Code :

```
#include <iostream> using
namespace std;
int matrixChainOrder(int p[], int size) {
    int n = size - 1;    int
    dp[100][100] = {0};
    for (int d = 1; d < n; d++) {

        for (int i = 1; i <= n - d; i++) {
            int j = i + d;
            dp[i][j] = 32767;          for (int
            k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (cost < dp[i][j]) {          dp[i][j] = cost;
                }
            }
        }
    }
    return dp[1][n];
}

int main() {
    int n;
    cout << "Enter the number of matrices: ";
    cin >> n;    int size = n + 1;    int p[100];
    cout << "Enter the " << size << " elements for the dimensions
    array:\n";
    for (int i = 0; i < size; i++) {
        cin >> p[i];
    }
    cout << "Minimum number of multiplications is: " <<
    matrixChainOrder(p, size) << endl;    return 0;
}
```



Output :

```
Enter the number of matrices: 5
Enter the 6 elements for the dimensions array:
6
5
7
3
1
9
Minimum number of multiplications is: 140
```

Time complexity :



Time Complexity:

```
for (int d=1; d<n-1; d++) {
    for (int i=1; i<n-d; i++) {
        j = i+d;
```

```
        for (int k=i; k<=j-1; k++) {
```

```
            q = C[i][k] + C[k+1][j] + P[i-1] * P[k] * P[j];
```

```
            if (q < min) {
```

```
                min = q; K[i][j] = k;
```

```
        C[i][j] = min; } }
```

$$\left[\frac{n(n-1)(n+1)}{6} \right]$$

$$T(n) = n + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + 4 \frac{n(n-1)(n+1)}{6} + \frac{n(n-1)}{2}$$

$$T(n) = O(n^3)$$

Best Case: Even in the best case, the algorithm checks all possible split

$$T(n) = n + n^2 + n^3 = O(n^3)$$

Average Case: on average case the algorithm also checks the all splits positions

$$T(n) = n + n^2 + n^3 = O(n^3)$$

Worst case: In this case all the positions are checked

$$T(n) = n + n^2 + n^3 = O(n^3)$$



Conclusion :

Thus, the Chain Matrix Multiplication problem was successfully implemented using Dynamic Programming, and the minimum number of scalar multiplications required was determined efficiently.

